

QuantShield Methodology

Author: Calvin J. Lomax

Abstract

QuantShield is a local portfolio research, model-training, and historical replay system that combines classical portfolio construction, benchmark-relative evaluation, and offline reinforcement learning inside a single desktop-oriented workflow. The repository couples a PySide6 application with a research stack for market-data ingestion, preprocessing, risk estimation, optimization, policy training, model scoring, and experiment management. This document formalizes the main theoretical and systems principles underlying the project, with emphasis on the mathematical structure of the optimization layer, the reward shaping used for policy learning, the replay engine used for historical simulation, and the software-engineering constraints required to make the system usable on commodity local hardware. The methodology draws on modern portfolio theory, covariance shrinkage, risk-parity allocation, actor-critic learning, offline RL, and transformer sequence modeling [1]-[8].

1. Problem Setting

Let

$$\mathcal{U} = \{1, 2, \dots, N\}$$

denote a user-selected universe of tradable assets, where $N \in \{10, 50\}$ for the currently supported desktop-facing model families. For each asset i , let $P_{i,t}$ denote the adjusted close price at business date t , and let the simple return be

$$r_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

At each rebalance date t , QuantShield constructs a portfolio weight vector

$$w_t \in \Delta^N$$

where

$$\Delta^N = \left\{ w \in \mathbb{R}^N : w_i \geq 0, \sum_{i=1}^N w_i = 1 \right\}.$$

Hence all supported strategies are long-only and fully invested. The practical desktop simulation then maps these continuous target weights into an integer-share executable portfolio subject to the prevailing asset prices and available capital.

The system addresses four related tasks:

1. estimate risk and expected return from historical data;
2. construct classical benchmark portfolios;
3. train policy models that allocate across arbitrary user-defined universes;
4. evaluate all strategies on matched historical replay windows.

2. Data Engineering

2.1 Market Data

The repository uses `yfinance` as the primary data source and caches downloaded price panels locally under `data/raw`. The data loader prefers exact cached files, then cached supersets, before attempting a live download. This design serves two purposes:

1. it minimizes repeated data acquisition cost;
2. it allows most of the app and test workflows to run locally without depending on network availability.

2.2 Cleaning and Alignment

Given a raw price panel P , the preprocessing layer:

1. normalizes the datetime index;
2. optionally forward-fills missing prices;
3. removes all-NaN assets when configured to do so;
4. converts prices into return panels.

The resulting return matrix

$$R \in \mathbb{R}^{T \times N}$$

is the common substrate for both classical optimization and offline RL dataset construction.

2.3 Universe Curation

The desktop app exposes canonical universes, saved portfolios, and preset portfolios. These lists are not purely conceptual. They are constrained by locally available historical coverage, because offering a static ticker list with no usable data creates systematic replay and training failures. The current preset and default universes are therefore curated against the repo’s available cached price history.

3. Classical Portfolio Construction

QuantShield uses several standard portfolio-construction baselines.

3.1 Equal Weight

The equal-weight portfolio for a universe of size N is a simple diversification baseline [9]:

$$w^{\text{EW}} = \left(\frac{1}{N}, \dots, \frac{1}{N} \right).$$

This serves as both a benchmark and a control. It is intentionally simple, transparent, and robust to estimation error.

3.2 Mean-Variance / Markowitz

For an estimated mean vector $\mu \in \mathbb{R}^N$ and covariance matrix $\Sigma \in \mathbb{R}^{N \times N}$, the long-only mean-variance problem is modeled as the Markowitz objective [1]. The covariance estimate may use shrinkage methods to improve conditioning in high-dimensional or short-history settings [2]:

$$\max_{w \in \Delta^N} \mu^\top w - \frac{\lambda}{2} w^\top \Sigma w$$

where $\lambda > 0$ is a risk-aversion parameter. In equivalent minimization form:

$$\min_{w \in \Delta^N} \frac{\lambda}{2} w^\top \Sigma w - \mu^\top w.$$

The desktop app’s replay comparison uses a rolling long-only Markowitz baseline constructed on the same rebalance dates as the model portfolio. This is important: the comparison is not merely against a static theoretical frontier, but against an implementable schedule-aligned allocation process.

3.3 Minimum Variance and Risk Parity

The research stack also uses minimum-variance and risk-parity objectives to generate comparison baselines and target policies.

Minimum variance solves

$$\min_{w \in \Delta^N} w^\top \Sigma w.$$

Risk parity seeks weights such that each asset contributes approximately the same amount of marginal portfolio risk [3]. If

$$\sigma_p(w) = \sqrt{w^\top \Sigma w},$$

then the contribution of asset i to portfolio risk is

$$\text{RC}_i(w) = w_i \frac{(\Sigma w)_i}{\sigma_p(w)}.$$

Risk parity aims to make $\text{RC}_i(w)$ as equal as feasible subject to the long-only simplex constraint.

4. Historical Replay Engine

The replay layer is a practical implementation layer rather than a purely analytical backtest abstraction.

4.1 Rebalance Schedule

Given a date index and a rebalance frequency f , the engine builds a rebalance schedule

$$\mathcal{T}_{\text{rebalance}} = \{t_1, t_2, \dots, t_K\}$$

using business-day, weekly, biweekly, or month-end spacing. The desktop app also supports auto-adjusted intervals designed to keep the number of decisions approximately comparable across different replay horizons.

4.2 Integer-Share Execution

A target continuous weight vector w_t is mapped into integer holdings. If total capital at date t is V_t and the price vector is p_t , then ideal dollar allocation is

$$a_t = V_t w_t.$$

Ideal shares would be a_t/p_t , but execution uses integer shares:

$$q_{i,t} = \left\lfloor \frac{a_{i,t}}{p_{i,t}} \right\rfloor.$$

Residual cash is retained explicitly. This removes the unrealistic fractional-share assumption and makes the historical simulation closer to an implementable retail execution model.

4.3 Portfolio Path

Given holdings q_t and subsequent prices $p_{t+\tau}$, portfolio value evolves as

$$V_{t+\tau} = \sum_{i=1}^N q_{i,t} p_{i,t+\tau} + c_t,$$

where c_t is residual cash after the last rebalance. Benchmark, equal-weight, and Markowitz comparison paths are produced on the same replay grid so all reported path-level metrics are directly comparable.

5. Offline Reinforcement Learning Formulation

5.1 State Representation

For a lookback window L , the policy state at decision time t is a tensor of rolling asset features over the last L observations. Abstractly,

$$s_t \in \mathbb{R}^{N \times L \times F}$$

where F is the feature dimension per asset per time step. The current implementation focuses on return-derived signals and related state features aligned across assets.

5.2 Action Space

The actor outputs a simplex-constrained allocation. In the actor-critic models, this allocation is represented with a Dirichlet policy over the simplex [10]:

$$\pi_\theta(s_t) = w_t \in \Delta^N.$$

This preserves long-only, fully invested behavior at the policy level. For desktop-facing synthetic-slot models, the policy is trained on position slots rather than fixed ticker identities, allowing the learned architecture to be applied to arbitrary user-defined portfolios of the same width.

5.3 Offline Dataset Construction

Offline training samples are built from:

1. a return panel R ;
2. one or more target-weight histories derived from classical optimization rules;
3. forward holding-period segments between rebalance dates.

For each rebalance date t_k , QuantShield stores:

- the state s_{t_k} ;
- a target allocation a_{t_k} ;
- a realized forward reward over the holding segment $[t_k + 1, t_{k+1}]$.

This makes the training problem a hybrid between imitation learning and value-based policy improvement over a fixed historical dataset [6], [11].

6. Reward Design

6.1 Raw Portfolio Reward

If the forward segment is $\{t + 1, \dots, t + h\}$ and the policy allocation is w_t , then the segment cumulative return is

$$R_t^\pi = \prod_{\tau=1}^h (1 + r_{t+\tau}^\top w_t) - 1.$$

This is the raw reward term.

6.2 Benchmark-Relative Reward Components

QuantShield does not optimize only raw return. It also measures excess return versus multiple comparison baselines:

- benchmark ETF or benchmark ticker;
- equal weight;
- restricted-random allocation;
- Markowitz mean-variance.

For a comparison strategy b , the excess term is

$$\Delta_t^{(\pi, b)} = R_t^\pi - R_t^b.$$

6.3 Composite Reward

The composite training reward is a weighted combination

$$\mathcal{R}_t = \alpha_{\text{raw}} R_t^\pi + \alpha_{\text{bm}} \Delta_t^{(\pi, \text{bm})} + \alpha_{\text{ew}} \Delta_t^{(\pi, \text{ew})} + \alpha_{\text{rr}} \Delta_t^{(\pi, \text{rr})} + \alpha_{\text{mv}} \Delta_t^{(\pi, \text{mv})}.$$

Recent app-facing training flows default to a “best-of-selected” comparison mode in which the benchmark/equal-weight/Markowitz comparison is taken against the strongest active baseline:

$$R_t^* = \max \{ R_t^{\text{bm}}, R_t^{\text{ew}}, R_t^{\text{mv}} \},$$

and the comparison term becomes

$$\Delta_t^{(\pi, \star)} = R_t^\pi - R_t^*.$$

This makes the policy compete against the hardest selected comparator rather than collecting easy reward from weaker baselines.

6.4 Short-Horizon Robustness

For very short horizons such as one-month daily-frequency training, covariance estimation can become unstable when forward segments are too short. The implementation therefore falls back to equal-weight comparisons whenever the Markowitz or forward optimization inputs are not statistically well-posed. This is a deliberate bias toward robustness over noisy pseudo-precision.

7. Transformer Actor-Critic Architecture

QuantShield uses a cross-asset attention architecture to process multi-asset rolling windows. If $x_{i,t}$ denotes the feature sequence for asset i , the model embeds each asset trajectory and applies stacked attention blocks across the joint asset-state representation. The cross-asset encoder follows the transformer self-attention mechanism [7].

At a high level:

1. per-asset temporal features are embedded into a hidden space of dimension d ;
2. cross-asset attention layers model interactions among assets;
3. the actor head outputs simplex-constrained weights;
4. the critic head estimates state value.

The architecture is parameterized by:

- hidden dimension d ;
- number of attention heads H ;
- number of layers L ;
- dropout probability p .

Because PyTorch multi-head attention requires divisibility,

$$d \equiv 0 \pmod{H},$$

the repository now normalizes incompatible (d, H) combinations before model construction rather than failing late during training.

8. Training Objective

The actor-critic training objective blends reinforcement-style reward optimization with behavior cloning toward the offline target allocations. The value-learning component follows actor-critic principles [5], [8], while the imitation component follows the behavior-cloning idea of learning directly from demonstrated actions [11]. In stylized form:

$$\mathcal{L} = \mathcal{L}_{\text{actor}} + \beta\mathcal{L}_{\text{BC}} + \gamma\mathcal{L}_{\text{critic}} + \eta\mathcal{L}_{\text{entropy}}.$$

Here:

- $\mathcal{L}_{\text{actor}}$ aligns the policy with reward improvement;
- $\mathcal{L}_{\text{BC}}$ keeps the policy anchored to useful target allocations;
- $\mathcal{L}_{\text{critic}}$ fits value estimates;
- $\mathcal{L}_{\text{entropy}}$ discourages premature collapse.

The behavior-cloning coefficient is especially important in small-data or short-horizon settings, where pure policy optimization can overfit to noise. Optimization uses Adam or AdamW-style adaptive gradient methods [12], [13].

9. Model Selection and Scoring

QuantShield does not treat the last epoch as automatically optimal. Instead, candidate models are evaluated on benchmark summaries and aggregate score tables. Statistical outperformance uses one-sided t -style tests grounded in the classical Student test [14]. The key statistics include:

- mean raw return;
- mean excess return versus benchmark;
- mean excess versus Markowitz;
- significance tests on outperformance;
- composite score aggregates across train, validation, and all-sample splits.

The desktop app then surfaces these saved models with quality labels such as **Validated**, **Benchmark+**, and **Exploratory**. The selector’s **Updated** column is intentionally relative rather than absolute because the most actionable question for a user is recency, not raw timestamp formatting.

10. Compute Allocation on Local Hardware

QuantShield is designed for local execution on commodity hardware, including Apple Silicon laptops. Model training is implemented with PyTorch [15]. Before app-triggered training begins, the system evaluates:

- physical core count;
- logical core count;
- available and total RAM;
- hardware acceleration availability (`mps`, `cuda`, or `cpu`).

Hyperparameters such as batch size and experiment candidate-pool size are then capped according to the device profile. This is a pragmatic systems decision. The objective is not to maximize nominal search breadth at all costs, but to keep the run feasible, observable, and recoverable on the user’s machine.

11. Desktop-App Systems Design

11.1 Separation of Concerns

The desktop app does not re-implement training algorithms. Instead, the UI delegates execution to existing scripts through a dedicated service layer. This preserves a clean architecture:

- scripts remain the canonical training entry points;
- the app provides validation, orchestration, telemetry, and persistence;
- saved models are re-discovered through the same checkpoint service used for replay.

11.2 Asynchronous Execution

Training is launched asynchronously through Qt process management so the UI remains responsive. The monitor window streams:

- stdout and stderr;
- structured metric events;
- candidate-level progress;
- CPU and RAM utilization.

This is important because model fitting is materially long-running relative to the rest of the application.

11.3 Modal and Non-Modal Workflow

The `New Model` launcher is part of the selection workflow, but the training monitor is intentionally non-modal. Once training begins, the launcher hides and the monitor continues independently so the rest of the desktop app remains interactive. When the monitor closes after completion, failure, or cancellation, the launcher is restored and the user can inspect, save, or discard the run.

12. Testing and Reliability

The repository includes unit and desktop-integration tests covering:

- data loading and schedule generation;
- optimization and risk routines;
- RL dataset construction and training utilities;
- desktop model selection, replay, charting, and training dialogs.

The project also favors defensive fallbacks over brittle failure in several places:

- unreadable checkpoint files are skipped during discovery instead of aborting app startup;
- short-horizon covariance estimation falls back to equal weight when necessary;
- external ticker benchmarks are added to the return panel without polluting the policy action space;
- invalid attention-head configurations are normalized before model construction.

These choices are not incidental. They are central to making a local quantitative research application usable in practice.

13. Conclusion

QuantShield is best understood as a layered decision-support system rather than a single algorithm. Classical optimization, offline policy learning, integer-share replay, desktop visualization, and experiment management all interact. The repository’s methodology therefore combines:

1. standard portfolio theory for transparent baselines;
2. reward-shaped offline RL for adaptive allocation policies;
3. systems-level safeguards for local execution and reproducibility;
4. UI design that exposes model behavior rather than hiding it.

The resulting application is not merely a charting wrapper around saved checkpoints. It is a full local environment for constructing, evaluating, training, and comparing portfolio allocation strategies under explicit mathematical and software-engineering constraints.

References

- [1] H. Markowitz, “Portfolio selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77-91, Mar. 1952.
- [2] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365-411, Feb. 2004.
- [3] S. Maillard, T. Roncalli, and J. Teiletche, “The properties of equally weighted risk contribution portfolios,” *The Journal of Portfolio Management*, vol. 36, no. 4, pp. 60-70, Summer 2010.
- [4] W. F. Sharpe, “Mutual fund performance,” *The Journal of Business*, vol. 39, no. 1, pp. 119-138, Jan. 1966.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [6] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” arXiv:2005.01643, 2020.
- [7] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [8] V. Mnih *et al.*, “Asynchronous methods for deep reinforcement learning,” in *Proc. 33rd Int. Conf. Machine Learning*, New York, NY, USA, 2016, pp. 1928-1937.
- [9] V. DeMiguel, L. Garlappi, and R. Uppal, “Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?” *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915-1953, May 2009.
- [10] T. P. Minka, “Estimating a Dirichlet distribution,” Microsoft Research, Cambridge, U.K., Tech. Rep., 2000.
- [11] D. A. Pomerleau, “ALVINN: An autonomous land vehicle in a neural network,” in *Advances in Neural Information Processing Systems*, vol. 1, 1989.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations*, 2015.
- [13] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations*, 2019.
- [14] Student, “The probable error of a mean,” *Biometrika*, vol. 6, no. 1, pp. 1-25, Mar. 1908.
- [15] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.